

Trabajo Práctico Integrador: Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

Asignatura: Desarrollo de Software

Modalidad: en grupo

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs). Sin embargo, el práctico está diseñado para evaluar el **razonamiento crítico, el análisis de estados de memoria y la justificación técnica**. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega

- Entregar un informe en formato PDF publicado en un repositorio de GitHub.
- Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas solicitadas.

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;

class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {
```

```

public static void main(String[] args) {
    Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();[cite: 3]
    TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

    tarjetasValidas.add(t1);

    // Modificación posterior a la inserción
    t1.setNivelAcceso(5);

    System.out.println("¿Tarjeta válida en sistema?: " +
tarjetasValidas.contains(t1));
    System.out.println("Cantidad de tarjetas registradas: " +
tarjetasValidas.size());
}
}

```

Responder y Justificar:

1. **Traza del Estado Interno:** Explique paso a paso qué ocurre dentro de la estructura HashSet:
 - Al ejecutar tarjetasValidas.add(t1):
Al insertar la tarjeta, la colección llama a su método hashCode(). Usando los datos iniciales (código "ACC-101" y nivel 2), se genera un número específico. El HashSet usa este número para determinar en qué "cajón" o posición de memoria guardar el objeto.
 - Al ejecutar t1.setNivelAcceso(5):
Al modificar el nivel de acceso, el valor que devolvería el hashCode() cambia. El problema es que el HashSet no detecta esta modificación, por lo que la tarjeta permanece almacenada en la ubicación asignada originalmente (la calculada con el nivel 2).
 - Al ejecutar tarjetasValidas.contains(t1):

Para buscar la tarjeta, Java calcula nuevamente el hashCode() utilizando el estado actual (nivel 5). Este nuevo cálculo arroja un número distinto, lo que hace que el HashSet busque en el cajón equivocado. Al no encontrarla allí, devuelve false, aunque el objeto siga existiendo dentro de la colección.

2. **Explicación del Cajón (Bucket):** ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer contains()?

El objeto quedó almacenado en el cajón (bucket) calculado a partir de sus valores originales ("ACC-101", nivel 2). Posteriormente, al ejecutar contains(), Java busca el elemento en el cajón correspondiente a los valores actuales ("ACC-101", nivel 5). Como los cálculos generan índices diferentes, la colección intenta ubicar la referencia en una posición incorrecta.

3. **Fuga de Memoria (Memory Leak):** ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

Este patrón genera una fuga de memoria lógica. Al alterarse el estado del objeto, este se vuelve inalcanzable para los métodos de búsqueda o eliminación (como remove()). En un servidor que opera de forma continua, si el sistema agrega tarjetas, modifica sus niveles de acceso y luego intenta borrarlas sin éxito, la colección crecerá de manera indefinida acumulando objetos "fantasma" que no se pueden liberar, lo que terminará consumiendo toda la memoria y provocando un error de tipo OutOfMemoryError en la máquina virtual de Java.

4. **Refactorización Obligatoria:** Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

Solución A: Inmutabilidad Absoluta Se eliminan los métodos mutadores (*setters*) y se declaran los atributos como final. Si un empleado cambia de nivel de acceso, la arquitectura debe forzar la instanciación de una nueva tarjeta en lugar de reciclar la existente.

```
class TarjetaAcceso {
    private final String codigo;
    private final int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }
    // equals() y hashCode() se mantienen idénticos.
}
```

Solución B (Ajuste de Identidad): Se modifica el criterio de identidad de la clase. El nivel de acceso pasa a considerarse un dato temporal y se excluye de los métodos equals() y hashCode(), haciendo que ambos dependan exclusivamente del atributo único e inmutable (codigo).

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    TarjetaAcceso tarjeta = (TarjetaAcceso) o;
    // La comparación ignora el nivelAcceso
    return Objects.equals(codigo, tarjeta.codigo);
}
```

```
@Override
public int hashCode() {
    // El hash se calcula únicamente con el código
    return Objects.hash(codigo);
}
```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de *E-Commerce*. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (java.util) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones

Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago

Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos

Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Responder y Justificar:

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O

Requerimiento	Interfaz Seleccionada (List , Set , Map , Queue) PDF	Implementación Concreta (ArrayList , TreeSet , HashMap , etc.) PDF	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría					
2. Ofertas					
3. Sesiones					

Requerimiento	Interfaz Seleccionada (List, Set, Map, Queue	Implementación Concreta (ArrayList, TreeSet, etc.	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría	List	ArrayList	$O(n)$	$O(1)$ amortizado	Asignación contigua en memoria. Optimiza el recorrido secuencial exigiendo minimizando los saltos de caché en el procesador.
2. Ofertas	Set/NavigableSet	TreeSet	$O(\log n)$	$O(\log n)$	Implementa internamente un árbol Rojo-Negro. Mantiene el ordenamiento natural de los nodos en tiempo real sin requerir ordenamiento posterior, permitiendo extraer los extremos (raíz/hojas) eficientemente
3. Sesiones	Map	HashMap	$O(1)$	$O(1)$	Estructura de tabla de dispersión. Al transformar el Token (String) en un índice de arreglo mediante la función hash, logra acceso en tiempo constante ideal para concurrencia masiva sin necesidad de preservar orden.

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) {  
    // Algoritmo ineficiente  $O(n^2)$   
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        boolean duplicado = false;  
        for (int j = 0; j < texto.length(); j++) {  
            if (i != j && c == texto.charAt(j)) {  
                duplicado = true;  
                break;  
            }  
        }  
        if (!duplicado) return c;  
    }  
    return null;  
}
```

Responder y Justificar:

1. **Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

El método original tiene una complejidad de $O(n^2)$ por los ciclos anidados. Para reducirlo a $O(n)$, se debe utilizar un LinkedHashMap. Esta estructura es la adecuada porque preserva el orden en el que se insertan los caracteres y permite inserciones y búsquedas en tiempo $O(1)$.

```
import java.util.LinkedHashMap;  
import java.util.Map;
```

```
public static Character primerCaracterNoRepetidoOptimo(String texto) {  
    Map<Character, Integer> frecuencias = new LinkedHashMap<>();  
  
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);  
    }  
  
    for (Map.Entry<Character, Integer> entry : frecuencias.entrySet()) {  
        if (entry.getValue() == 1) {  
            return entry.getKey();  
        }  
    }  
  
    return null;  
}
```

Al realizar dos recorridos secuenciales independientes en lugar de anidados, la complejidad total se mantiene en $O(n)$.

2. **Análisis de Colisiones:** Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Si dos caracteres distintos devuelven el mismo hashCode(), se produce una colisión. El HashMap no pierde la información; guarda ambos elementos en el mismo cajón (bucket) utilizando una lista enlazada. Al momento de buscar, localiza el cajón por el hash y luego itera sobre los elementos guardados allí usando el método equals() para comparar el valor real del carácter y encontrar la coincidencia exacta.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que `s1.hashCode() == s2.hashCode()` devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un HashSet<String>, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos hashCode() y equals() internamente para determinar si s2 debe ingresarse o desecharse.

El HashSet contendrá exactamente dos elementos. Al intentar insertar "Ea", Java calcula su hashCode() y va al mismo cajón de memoria donde ya está "FB" (ya que ambos devuelven 2236). Para resolver esta colisión y verificar si es un elemento duplicado, Java utiliza el método equals() ("FB".equals("Ea")). Como devuelve false, valida que son cadenas distintas y admite la inserción del segundo objeto.

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un HashMap en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

La acumulación de colisiones degrada el rendimiento óptimo de $O(1)$ del HashMap. En el peor escenario posible, si todos los elementos caen en el mismo cajón, se forma una lista enlazada y el tiempo de búsqueda decae a $O(n)$. Sin embargo, desde Java 8, la estructura se optimiza dinámicamente: si un cajón supera un límite de colisiones (8 elementos), la lista enlazada se transforma en un árbol binario balanceado (Red-Black Tree). Esto limita la penalización de rendimiento, asegurando que el peor caso sea de $O(\log n)$.

